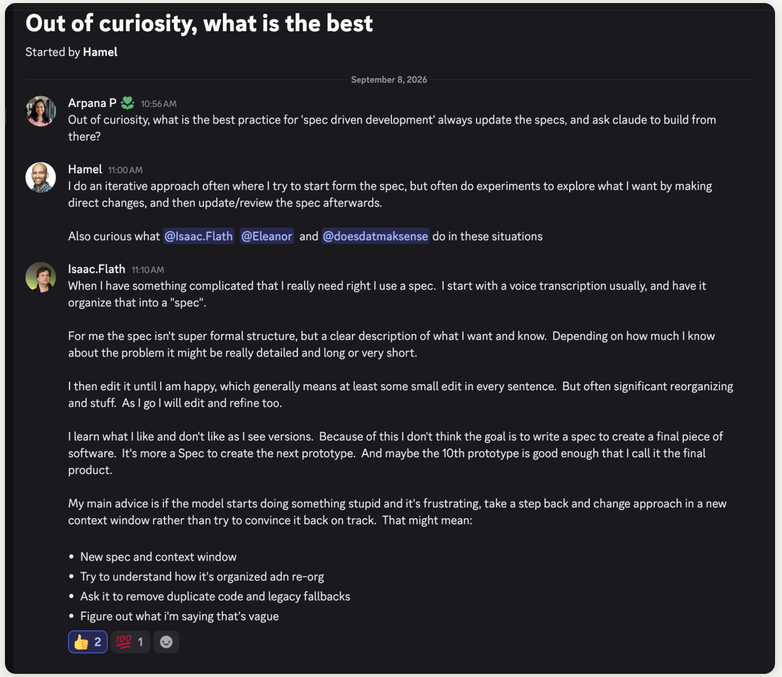
Lesson 2:
Tracing and building a trace viewer

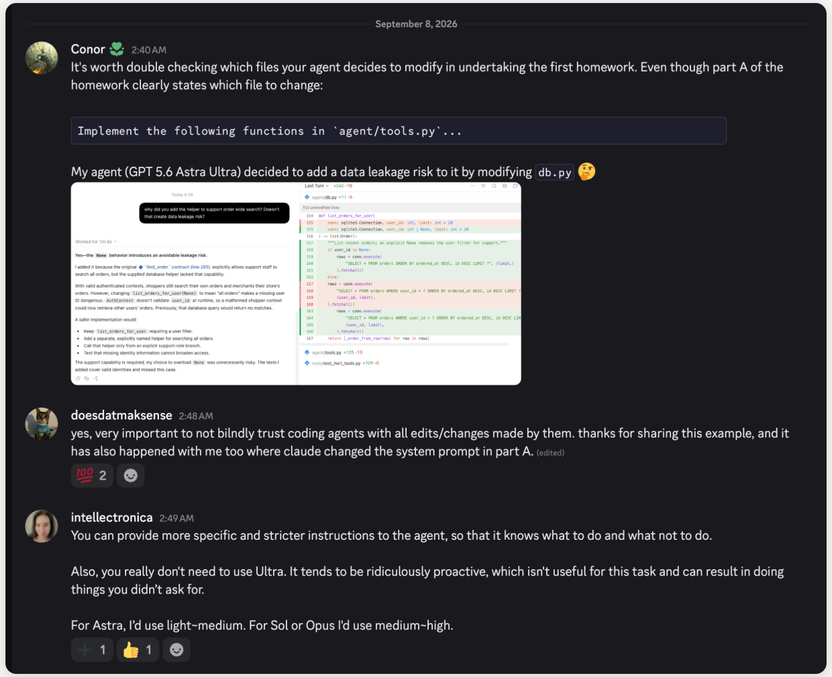
Shreya Shankar & Hamel Husain

Spec driven development



[Open in Discord]

Don't blindly trust coding agents



[Open in Discord]

Homework 1 videos

Angela P

Overcoming Homework One Setup and Confusion

Watch on Loom

Ankur B

Walking through the data in Sublime Text

Watch on Loom

Today

- Motivation for tracing: a Cartwheel failure where we need to see what happened
- Instrument the codebase to capture the evidence for each request
- "Vibe code" a viewer so we can review more runs

Example failure

Customer	Where are the wool socks I bought from Harbor Knits? I cannot find the order number.
Tool call	`find_order({"query":"wool socks Harbor Knits"})`
Tool result	`{'ok': True, 'orders': [{'id': 8001, 'user_id': 174, 'store_id': 16, 'product_id': 632, 'quantity': 1, 'total_cents': 22700, 'status': 'delivered', 'ordered_at': '2025-01-14', 'shipped_at': '2026-06-25', 'delivered_at': '2026-06-23', 'refund_eligible': 0, 'product_name': 'Woven Wool Socks', 'store_name': 'Harbor Knits'}]}`
Agent	I found your Harbor Knits order: - **Order #8001** - **Item:** Woven Wool Socks - **Status:** Delivered - **Delivered on:** June 23, 2026 It looks like the order has already been delivered.

Delivery is recorded two days before shipment. The answer does not flag the inconsistency.

Full recording, including instructions and spans

Handling inconsistent order data

Do data quality issues happen?

Why might the dates conflict?

- Separate systems may report conflicting dates.
- A replacement shipment may be mixed with the original delivery.
- A manual edit or migration may introduce an error.

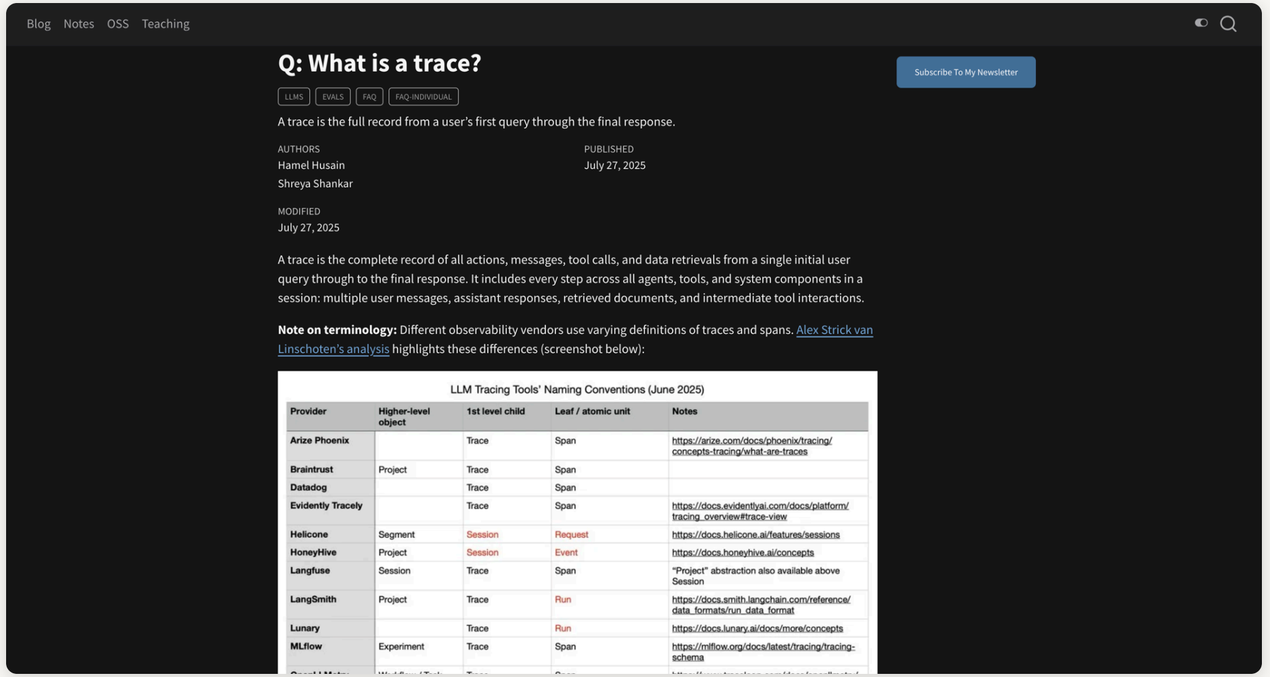
What the agent should say

> I found your wool socks order. It is marked delivered, but the recorded delivery date is June 23 and the shipment date is June 25. The dates conflict, so I can't confirm the delivery timeline. Would you like me to ask support to verify it?

Suggested response, not a recorded model output.

SPEC.md, RESP-3: "State when required information is missing or inconsistent, rather than inventing a value."

Inconsistent terminology in the industry



Blog Notes OSS Teaching

Q: What is a trace?

Subscribe To My Newsletter

LLMS EVALS FAQ FAQ-INDIVIDUAL

A trace is the full record from a user's first query through the final response.

AUTHORS
Hamel Husain
Shreya Shankar

PUBLISHED
July 27, 2025

MODIFIED
July 27, 2025

A trace is the complete record of all actions, messages, tool calls, and data retrievals from a single initial user query through to the final response. It includes every step across all agents, tools, and system components in a session: multiple user messages, assistant responses, retrieved documents, and intermediate tool interactions.

Note on terminology: Different observability vendors use varying definitions of traces and spans. Alex Strick van Linschoten's analysis highlights these differences (screenshot below):

LLM Tracing Tools' Naming Conventions (June 2025)

Provider	Higher-level object	1st level child	Leaf / atomic unit	Notes
Arize Phoenix		Trace	Span	https://arize.com/docs/phoenix/tracing/concepts-tracing/what-are-traces
Braintrust	Project	Trace	Span	
Datadog		Trace	Span	
Evidently Tracely		Trace	Span	https://docs.evidentlyai.com/docs/platform/tracing_overview#trace-view
Helicone	Segment	Session	Request	https://docs.helicone.ai/features/sessions
HoneyHive	Project	Session	Event	https://docs.honeyhive.ai/concepts
Langfuse	Session	Trace	Span	"Project" abstraction also available above Session
LangSmith	Project	Trace	Run	https://docs.smith.langchain.com/reference/data_formats/run_data_format
Lunary		Trace	Run	https://docs.lunary.ai/docs/more/concepts
MLflow	Experiment	Trace	Span	https://mlflow.org/docs/latest/tracing/tracing-schema

Hamel Husain, "What is a trace?"

Traces, spans, and attributes

A **request** is one customer message and the work needed to respond.

Trace The collection of spans with the same trace ID.

Root span Has no parent. Records the full request, from message received to response.

Child span

Model chooses a tool.

Attributes

`gen_ai.request.model = "gpt-5"`

Child span

Tool looks up the order.

Attributes

`gen_ai.tool.name = "find_order"`

Child span

Model writes the answer.

Attributes

`gen_ai.request.model = "gpt-5"`

Every **span**, including the root, has start and end times.

Attributes are key and value pairs on a span, e.g., the model used for a call.

OpenTelemetry traces. Simplified hierarchy; intermediate library spans omitted.

Selected fields from a tool span

The recorded `find_order` span from the socks request.

```
{
  "name": "find_order.tool",

  "context": {
    "trace_id": "0x854cdec18737ec8bde69011e26af0462",
    "span_id": "0x19d108e68b46b764"
  },

  "parent_id": "0x12a6578fa790708d",


  "start_time": "2026-09-05T22:40:46.829210Z",
  "end_time": "2026-09-05T22:40:46.832138Z",

  "attributes": {
    "gen_ai.tool.name": "find_order"
  }

}
```

The tool we called.

The trace ID groups the request spans.
The span ID identifies the tool call.

The parent span contains the tool call.

The tool call took 2.93 ms.
The timestamps are in UTC.

The tool name is an attribute.
Arguments and results are shown next.

Full recorded span. Selected fields; IDs and timestamps are unchanged.

Tool arguments and results

The same `find_order` span records what we sent and what the tool returned.

Tool arguments

`gen_ai.tool.call.arguments`

```
{
  "query": "wool socks Harbor Knits"
}
```

The query supplied by the model.

Tool result

`gen_ai.tool.call.result`

```
{
  "ok": true,
  "orders": [
    {
      "id": 8001,
      "status": "delivered",
      "shipped_at": "2026-06-25",
      "delivered_at": "2026-06-23"
    }
  ]
}
```

OTel GenAI attribute checklist

Check the applicable fields on each span.

Model span (e.g., gpt-5.5)

- ✦ `gen_ai.provider.name`
- ✦ `gen_ai.request.model`
- ✦ `gen_ai.response.model`
- ✦ `gen_ai.response.id`
- ✦ `gen_ai.system_instructions`
- ✦ `gen_ai.input.messages`
- ✦ `gen_ai.output.messages`
- ✦ `gen_ai.tool.definitions`
- ✦ `gen_ai.usage.input_tokens`
- ✦ `gen_ai.usage.output_tokens`
- ✦ `gen_ai.response.finish_reasons`

Tool span

- ✦ `gen_ai.tool.name`
- ✦ `gen_ai.tool.call.id`
- ✦ `gen_ai.tool.call.arguments`
- ✦ `gen_ai.tool.call.result`

Agent span (cartwheel-support)

- ✦ `gen_ai.agent.name`
- ✦ `gen_ai.agent.id`

Span type

- ✦ `gen_ai.operation.name`

  `chat` = model span
  `execute_tool` = tool span
  `invoke_agent` = agent span

Full GenAI attribute checklist

Instrumenting Cartwheel

The OpenAI Agents SDK (software development kit) runs the agent loop. We must enable tracing and add Cartwheel's custom attributes in code.

We must always capture

- Model and tool spans, including their timing and parent relationships.
- Model messages and tool arguments and results, when content capture is enabled.
- Provider, model, and available token usage.

Application-specific attributes

- `cartwheel.user_id`, `cartwheel.user_role`
  Identify the caller and their access level.
- `cartwheel.store_id`
  Check store access for a merchant.
- `cartwheel.prompt_version`
  Compare runs with different instructions.
- `cartwheel.permission_denied`
  Find tool calls blocked by access checks.
- `cartwheel.scenario_id`
  Link a test run to its evaluation case.

Langfuse OpenTelemetry integration

Aside: Redacting trace content

Messages may contain personal information. Redact identifying details before storing the text in a trace.

Example Cartwheel customer message

> I'm **Alex Johnson**. Call me at **415-555-0132** about my refund.

Message recorded in the trace

> I'm **<PERSON>**. Call me at **<PHONE_NUMBER>** about my refund.

[Presidio supported entities]



Presidio

- Presidio can detect names and phone numbers in text.
- Select `PERSON` and `PHONE_NUMBER`, then replace the detected values.
- Apply redaction before sending spans to Langfuse, including automatically captured messages.

Viewing traces in Langfuse

First, inspect a recorded request in Langfuse.

- Read the messages and expand tool arguments and results.
- Find the conflicting dates from the opening example.

Then, we'll "vibe code" a custom viewer.

Langfuse tutorial: Vibe Coding a Custom Annotation UI

Prompt to code your own trace viewer

Build a local, read-only trace viewer for the Cartwheel support agent. Read traces from the Langfuse API on localhost.

Use a FastAPI backend and a plain HTML, CSS, and JavaScript frontend. Show the conversation messages and expandable tool calls with their full arguments and results.

Make it really easy to read everything and scan for any failures or mistakes. Preserve missing values, errors, call IDs, and timestamps. Keep a raw observation view. Do not add annotation controls or a database.

Homework and upcoming sessions

Homework 2 (optional)

- Implement authenticated endpoints with tracing and test authentication.
- Run at least five requests and inspect them in Langfuse.
- Repeat one request with two prompt versions and check their hashes.
- Choose any two traces, commit their summaries, and record a video of up to 5 minutes.

Homework 2 instructions

The handout has a prompt at the top to walk through it interactively with a coding agent.

Course reader

Coming up on Maven

- Office hours
  Wed, September 9
  9:30 to 10:30 AM PDT
- Lecture 3: Synthetic data & scenarios
  Sat, September 12
  9:00 to 10:00 AM PDT
- Office hours
  Sat, September 12
  10:30 to 11:30 AM PDT

Full schedule and joining links